

Technical Appendix: SBayesRC-Style Annotation Priors

Peter Sørensen

Table of contents

1	Model Definition	2
2	Optional BayesS-Style selection_s	3
3	Gamma Grids	4
4	Annotation-Dependent Component Probabilities	4
4.1	Why the probabilities sum to one	5
4.2	Numerical flooring and renormalization	5
5	Annotation Matrix and Preprocessing	5
5.1	Intercept prior	6
6	Alpha and SigmaSqAlpha Priors	6
7	Gibbs-Style Alpha Updating	6
8	Initialization	7
9	Marker-Effect and Component Updates	7
10	Fit Outputs	8
10.1	alpha	8
10.2	sigmaSqAlpha	8
10.3	comp_prob	8
10.4	ncomp	8
11	Diagnostic Helpers	9
11.1	Annotation-row diagnostics	9
11.2	Marker-profile diagnostics	9
12	Interpretation of Annotation Effects	10
13	Comparison With Published SBayesRC	10

This appendix describes the SBayesRC-style annotation-dependent BayesR prior available through `stblr_csr_annot(annotation_model = "sbayesrc")` and the compatibility wrapper `stblr_csr_sbayesrc_generic()`. The core prior family is closely aligned with published SBayesRC: marker effects follow a multi-normal mixture, and marker-specific mixture probabilities are determined by conditional probit stick-breaking regressions on annotations. The exposed `sblr` model is not an exact reproduction of the published GCTB SBayesRC implementation. Its default gamma grid differs, and it uses a CSR sparse-LD likelihood rather than the GCTB low-rank rotated summary-statistics likelihood.

This note focuses on the annotation-dependent prior and its diagnostics. It does not claim computational equivalence between the two likelihood implementations.

1 Model Definition

For marker i , let $z_i \in \{1, \dots, C\}$ identify one of C mixture components. Conditional on $z_i = k$, the marker effect is

$$\beta_i \mid z_i = k, v_b, \gamma \sim N(0, \gamma_k v_b).$$

The vector

$$\gamma = (\gamma_1, \dots, \gamma_C)$$

contains fixed variance multipliers, while v_b is the sampled global active-effect variance scale. The implementation requires

$$\gamma_1 = 0, \quad \gamma_k > 0 \quad \text{for } k > 1.$$

The first component is therefore an exact null component:

$$\beta_i = 0 \quad \text{when } z_i = 1.$$

The remaining components represent increasingly diffuse active-effect distributions when their gamma multipliers increase.

For multiple traits supplied to the wrapper, this mixture construction and its annotation coefficients are fitted separately by trait. The formatted fit therefore stores one `alpha` matrix and one set of marker component summaries per trait.

2 Optional BayesS-Style selection_s

The package argument is `selection_s`. It corresponds to the BayesS-style MAF architecture parameter `S`.

Current CSR SBayesRC supports both fixed global `selection_s` and sampled trait-specific `selection_s` by Metropolis-Hastings. SBayesRC annotations and `selection_s` affect different parts of the prior:

- annotations affect marker-specific component probabilities through $A\alpha$ and probit stick-breaking;
- `selection_s` affects marker-specific effect-size prior variance.

CSR effects are on the standardized-genotype scale. The MAF-dependent prior scale is

$$q_j(S) = h_j^{S+1}, \quad h_j = 2p_j(1 - p_j),$$

where p_j is the minor allele frequency. The +1 exponent is used because the CSR sampler effects are standardized-genotype-scale effects rather than allele-scale effects.

With fixed `selection_s`, the active SBayesRC component prior is

$$b_j \mid \text{component}_j = m, v_b, S \sim N\{0, v_b \gamma_m q_j(S)\}.$$

With sampled `selection_s`, the current implementation samples one S_t per trait and chain using the active-component MH log posterior contribution

```
log p(S_t | b_t, gamma_t, vb_t)
= log p(S_t)
  - 0.5 sum_{j: gamma_jt > 0} [
    log q_j(S_t) + b_jt^2 / (vb_t gamma_jt q_j(S_t))
  ]
```

The sampled-S defaults are:

```
selection_s_init = 0
selection_s_prior = c(-3, 2)
selection_s_proposal_sd = 0.35
```

The same fixed and sampled `selection_s` support is available for plain CSR BayesC and plain CSR BayesR. Scheduled CSR BayesC, BayesC-like annotation-aware CSR backends, and BED backends do not support sampler-level `selection_s`. See [selection_s implementation status](#) for the complete backend support matrix and BayesC/BayesR kernels.

3 Gamma Grids

The public `sblr` default is:

```
gamma = c(0, 0.01, 0.1, 1)
```

Published SBayesRC uses the five-component grid:

```
gamma = c(0, 0.001, 0.01, 0.1, 1)
```

Users can request that published gamma grid explicitly:

```
fit <- stblr_csr_annot(  
  stats = stats,  
  Glist = Glist,  
  annotations = A,  
  annotation_model = "sbayesrc",  
  gamma = c(0, 0.001, 0.01, 0.1, 1)  
)
```

Matching the gamma grid aligns this part of the prior specification. It does not make the complete `sblr` model identical to GCTB SBayesRC because the likelihood, sparse-LD representation, exposed defaults, and implementation details remain distinct.

4 Annotation-Dependent Component Probabilities

Let A be an $m \times K$ marker-by-annotation matrix. For C mixture components, let α be a $K \times (C-1)$ coefficient matrix. Column α_j controls conditional stick-breaking step j .

For marker i and step j , the linear predictor and conditional continuation probability are

$$\eta_{ij} = A_i \alpha_j, \quad p_{ij} = \Phi(\eta_{ij}),$$

where Φ is the standard normal distribution function. The marker-level component probabilities are

$$\begin{aligned} \pi_{i1} &= 1 - p_{i1}, \\ \pi_{i2} &= p_{i1}(1 - p_{i2}), \\ \pi_{i3} &= p_{i1}p_{i2}(1 - p_{i3}), \\ &\vdots \\ \pi_{iC} &= \prod_{j=1}^{C-1} p_{ij}. \end{aligned}$$

Equivalently, component $k < C$ receives the probability of continuing through all earlier steps and stopping at step k ; the final component receives the probability of continuing through every step.

4.1 Why the probabilities sum to one

For a marker with step probabilities $p_1, \dots, p_{C-1}$,

$$\sum_{k=1}^C \pi_k = (1 - p_1) + p_1(1 - p_2) + \dots + \left(\prod_{j=1}^{C-2} p_j \right) (1 - p_{C-1}) + \prod_{j=1}^{C-1} p_j = 1.$$

The expression telescopes: probability either stops at one of the first $C - 1$ steps or reaches the final component.

4.2 Numerical flooring and renormalization

The native sampler computes these probabilities from $A\alpha$, floors each component probability at `pi_floor`, and renormalizes each marker row to sum to one. The public default is `pi_floor = 1e-12`.

Flooring prevents exact numerical zero probabilities when probit predictors are extreme. Because flooring slightly changes the unnormalized stick-breaking probabilities, the subsequent row renormalization is required. This is a numerical-stability adjustment, not a different conceptual link function.

The deterministic R diagnostic helpers apply the mathematical stick-breaking transform without the native sampler's floor-and-renormalize step. For ordinary finite coefficients, their rows sum to one up to numerical tolerance.

5 Annotation Matrix and Preprocessing

Rows of A must be aligned with the markers in the sufficient statistics, CSR sparse-LD matrix, and posterior outputs. Columns may contain binary or quantitative annotations, and overlap is allowed.

Before fitting, `stblr_csr_sbayesrc_generic()` calls the shared annotation preparation helper using these controls:

- `add_intercept = TRUE` adds an all-ones first column when one is not already recognized;
- `standardize_annotations = TRUE` centers and scales nonbinary quantitative annotations;
- `center_binary_annotations = FALSE` leaves binary columns unchanged by default;
- row names can be used to reorder A to known marker names when all names are available.

The preprocessing choices determine the meaning and scale of `alpha`. Quantitative coefficients are effects per standardized annotation unit under the defaults. An unchanged binary coefficient is a contrast associated with membership in that annotation, conditional on the remaining columns.

5.1 Intercept prior

With `intercept_flat = TRUE`, the first annotation coefficient at every stick-breaking step receives a flat prior in the native alpha update. This assumes the first column is the intercept, which is the arrangement produced when the wrapper adds it. All other annotation coefficients receive the normal prior described below.

If a custom matrix already contains an intercept, place it first when relying on `intercept_flat = TRUE`. An all-ones column elsewhere in the matrix is not treated as the flat-prior intercept by the native update.

6 Alpha and SigmaSqAlpha Priors

For non-intercept annotation coefficient k at stick-breaking step j , the implemented prior is

$$\alpha_{kj} \mid \sigma_{\alpha,j}^2 \sim N(0, \sigma_{\alpha,j}^2).$$

Each step has its own annotation-effect variance $\sigma_{\alpha,j}^2$, represented by `sigmaSqAlpha[j]`. Under `intercept_flat = TRUE`, the first coefficient is excluded from this normal shrinkage prior and from the variance update.

The variance update has scaled-inverse-chi-square or inverse-gamma family form:

$$\sigma_{\alpha,j}^2 = \frac{\sum_{k \in \mathcal{R}} \alpha_{kj}^2 + b_\alpha}{\chi_{|\mathcal{R}|+a_\alpha}^2},$$

where $\mathcal{R}$ contains coefficients subject to shrinkage, `sigmaSqAlpha_a` supplies a_α , and `sigmaSqAlpha_b` supplies b_α . Both public defaults are 2.

This construction is aligned with the published SBayesRC use of normal annotation-coefficient priors and a scaled-inverse-chi-square-family variance prior. The exact published hyperparameter specification should not be inferred from the `sblr` defaults. Users should report the values supplied to `sigmaSqAlpha_a`, `sigmaSqAlpha_b`, `sigmaSqAlpha_init`, and `intercept_flat`.

7 Gibbs-Style Alpha Updating

When `updateAlpha = TRUE`, the native sampler updates alpha every `alpha_update_every` iterations. The public default interval is 10.

The update uses conditional probit data augmentation. At stick-breaking step j , only markers that reached that step are included. A latent normal variable is sampled for each eligible marker:

$$\ell_{ij} \mid \alpha_j \sim N(A_i \alpha_j, 1),$$

truncated above zero when the marker continues beyond step j , and truncated at or below zero when it stops at that step.

Conditional on the latent variables and the other coefficients, each annotation coefficient has a normal full conditional. For a non-intercept coefficient, the prior precision $1/\sigma_{\alpha,j}^2$ is added to the data precision. With `intercept_flat = TRUE`, the first coefficient omits that prior precision. After the coefficients for a step are updated, `sigmaSqAlpha[j]` is drawn from the scaled-inverse-chi-square-family update.

This is a Gibbs-style conditional update using truncated-normal augmentation and normal draws. It is not a random-walk Metropolis-Hastings alpha update. After an alpha update, the native sampler recomputes all marker-specific prior component probabilities from $A\alpha$.

8 Initialization

`make_sbayesrc_alpha_init()` creates deterministic initial values for the annotation-dependent mixture architecture.

Given an initial overall active probability `pi_init`, it distributes active mass across the $C - 1$ non-null components using normalized `active_comp_weights`. This produces initial marginal component probabilities

$$\pi^{(0)} = (1 - \pi_{\text{init}}, \pi_{\text{init}} w_2, \dots, \pi_{\text{init}} w_C).$$

The helper then converts these marginal probabilities to conditional continuation probabilities for the stick-breaking representation. When A contains an all-ones column, the first such column receives intercept coefficients

$$\alpha_{0j}^{(0)} = \Phi^{-1}\{p_j^{(0)}\},$$

and all other coefficients begin at zero. If no all-ones column exists, all default coefficients are zero. Supplied `alpha_init` and `sigmaSqAlpha_init` values are accepted after dimension and finite-value validation.

The helper also returns named initial component probabilities, normalized active-component weights, conditional step probabilities, gamma values, and component names. These make initialization auditable without running the sampler.

9 Marker-Effect and Component Updates

For each marker, the sampler combines the annotation-derived prior component probabilities with the summary-statistics likelihood. The exact null component has $\gamma_1 = 0$ and leaves the marker effect at zero. For active component k , the effect prior variance is

$$v_{b,k} = \gamma_k v_b.$$

The sampler forms a component-specific Bayes factor using the current marker score, residual variance, marker diagonal cross-product, and $v_{b,k}$. It samples a new component from the resulting

posterior component probabilities, then samples the marker effect from its conditional normal distribution when the selected gamma is positive.

The global scale v_b may also be updated from currently active marker effects and their selected gamma values when `updateB = TRUE`. Thus gamma defines relative component scales, while v_b supplies the shared sampled scale.

10 Fit Outputs

The formatted fit exposes the standard marker and variance summaries together with SBayesRC-style architecture summaries.

10.1 alpha

`fit$alpha` is a list with one posterior-mean coefficient matrix per trait. Each matrix has annotations in rows and stick-breaking steps in columns. Because it is a posterior mean, applying a nonlinear probability transform to it gives a plug-in diagnostic, not the posterior mean of that transformed quantity.

10.2 sigmaSqAlpha

`fit$sigmaSqAlpha` is a trait-by-step matrix of posterior mean annotation-effect variance parameters. Large values permit more dispersion among non-intercept annotation coefficients at that stick-breaking step. They should be interpreted together with alpha coefficients, annotation scaling, and chain diagnostics.

10.3 comp_prob

`fit$comp_prob` is a list with one marker-by-component matrix per trait. It contains posterior frequencies of sampled component assignments after burn-in and thinning. It is not the same object as the annotation-derived prior probability matrix computed from $A\alpha$.

Component probability columns are named by gamma values. The exact null component column is `gamma_0.00`, so `dm = 1 - P(gamma_0.00)`. This differs from CSR BayesR, which uses `component_0` for the null component column.

10.4 ncomp

`fit$ncomp` is a trait-by-component matrix containing posterior mean counts of markers assigned to each component. Its row sum should equal the number of markers, subject only to numerical representation.

The usual outputs remain available when returned, including `bm`, `dm`, variance traces, covariance summaries, current effects, and residual state. Full marker-scale matrices should be retained for analysis rather than printed in full.

11 Diagnostic Helpers

Four deterministic exported helpers translate alpha coefficients into compact prior-architecture diagnostics.

11.1 Annotation-row diagnostics

`sbayesrc_annotation_pi(alpha, gamma)` applies the probit stick-breaking transform separately to each row of `alpha`. It returns an annotation-by-component probability matrix.

`sbayesrc_annotation_gamma_mean(alpha, gamma)` computes

$$\bar{\gamma}_k^{\text{annotation}} = \sum_{c=1}^C \pi_{kc} \gamma_c$$

for each alpha row.

These are coefficient-row diagnostics. A binary annotation row, for example, is evaluated as a standalone predictor row. It does not automatically include the fitted intercept or the effects of correlated annotations. It should not be described as the literal component probability for every marker carrying that annotation.

11.2 Marker-profile diagnostics

`sbayesrc_marker_pi(A, alpha, gamma)` first computes $A\alpha$, then applies the stick-breaking transform. It returns an $m \times C$ marker-level prior component probability matrix.

`sbayesrc_marker_gamma_mean(A, alpha, gamma)` computes the plug-in expected gamma for each marker:

$$\bar{\gamma}_i^{\text{marker}} = \sum_{c=1}^C \pi_{ic} \gamma_c.$$

These marker-profile helpers include the complete annotation combination in each row of A . They are generally more directly relevant to the fitted prior architecture than standalone annotation-row summaries.

```
alpha_t <- fit$alpha[[1]]
prior_pi <- sbayesrc_marker_pi(fit$input$A, alpha_t, fit$input$gamma)
expected_gamma <- sbayesrc_marker_gamma_mean(
  fit$input$A, alpha_t, fit$input$gamma
)
```

Because these diagnostics transform posterior mean alpha coefficients, they do not propagate the full posterior uncertainty in alpha. Compact reports can show annotation-row expected gamma, `summary(expected_gamma)`, and the correlation between marker expected gamma and PIP. That correlation describes the fitted prior-posterior relationship; it is not a calibration statistic or causal estimate.

12 Interpretation of Annotation Effects

An alpha coefficient changes a conditional continuation probability at one stick-breaking step. Its sign cannot be interpreted independently of component ordering:

- a positive coefficient increases the probability of continuing past that step;
- a negative coefficient increases the probability of stopping at that step;
- the consequences for expected gamma depend on all later steps and the gamma grid.

Expected gamma is a useful scalar summary of prior mixture allocation, but it compresses a complete component-probability vector. Two markers can have the same expected gamma while assigning probability to different components. Report component probabilities when those distinctions matter.

Annotation effects and expected-gamma differences are prior-architecture summaries, not proof of biological causality. They can reflect LD, marker density, annotation overlap, quantitative-annotation scaling, model misspecification, and finite-chain behavior. Interpretation should compare annotation-derived prior probabilities with posterior component frequencies and assess sensitivity to preprocessing, gamma grids, sparse-LD settings, and hyperparameters.

13 Comparison With Published SBayesRC

Published SBayesRC feature	<code>sblr</code> implementation	Assessment
BayesR-like multi-normal marker-effect mixture	$\beta_i \mid z_i = k \sim N(0, \gamma_k v_b)$	Same core prior family
Exact null component	Requires <code>gamma[1] == 0</code> ; null effects are exactly zero	Same
Published gamma grid <code>c(0, 0.001, 0.01, 0.1, 1)</code>	User-supplied and supported	Same when explicitly requested
Default gamma grid	<code>c(0, 0.01, 0.1, 1)</code>	Different exposed default
Marker-specific mixture probabilities from annotations	Uses $A\alpha$ for every marker	Same core structure
Conditional probit stick-breaking	Uses $p_{ij} = \Phi(A_i \alpha_j)$ and conditional continuation probabilities	Same core structure
Intercept and functional annotations	Adds an intercept by default; accepts overlapping binary or quantitative annotations	Closely aligned, with <code>sblr</code> preprocessing defaults

Published SBayesRC feature	sblr implementation	Assessment
Quantitative annotation handling	Standardizes nonbinary annotations by default; leaves binary columns unchanged by default	Qualified implementation choice
Normal alpha priors with step-specific variance	Non-intercept coefficients use $N(0, \sigma_{\alpha,j}^2)$	Same prior family
Scaled inverse-chi-square prior on alpha variance	Uses a scaled-inverse-chi-square/inverse-gamma-family update controlled by sigmaSqAlpha_a and sigmaSqAlpha_b	Same family; exact published hyperparameters not implied
Probit alpha update	Latent truncated-normal augmentation and conditional normal draws	Closely aligned Gibbs-style update
Component-probability numerical handling	Floors probabilities at pi_floor and renormalizes rows	Qualified numerical-stability detail
Summary-statistics likelihood	CSR sparse-LD likelihood	Different from GCTB low-rank rotated likelihood

The most accurate description is therefore:

`stblr_csr_sbayesrc_generic()` implements an SBayesRC-style, annotation-dependent BayesR prior that is closely aligned with the published SBayesRC prior structure, fitted with the **sblr** CSR sparse-LD likelihood.

It should not be described as an exact reproduction of published GCTB SBayesRC as exposed by default.

14 Practical Validation Checklist

Before interpreting an SBayesRC-style fit:

- verify marker alignment among A , sufficient statistics, CSR LD, and posterior summaries;
- record the gamma grid and distinguish the **sblr** default from the published grid;
- record intercept, standardization, and binary-centering choices;
- inspect annotation prevalence, scaling, overlap, and collinearity;
- report **sigmaSqAlpha_a**, **sigmaSqAlpha_b**, **intercept_flat**, **pi_floor**, and **alpha_update_every**;
- check component counts, PIPs, variance traces, and chain behavior;
- distinguish annotation-derived prior probabilities from posterior component frequencies in **comp_prob**;
- compare annotation-row and full marker-profile diagnostics;
- assess sensitivity to gamma, sparse-LD thresholds, annotations, and hyperparameters; and
- avoid causal claims from alpha coefficients or expected-gamma summaries.

15 Related Notes and Sources

- [Annotation-informed models](#) compares this model with fixed, learned, and group annotation priors.
- [Technical annotation priors](#) develops marker-specific inclusion and variance priors outside the BayesR mixture construction.
- [Workflows and outputs](#) explains compact fit and expected-gamma reporting.
- [Model overview](#) places the model within the broader `sblr` model and workflow landscape.

The implementation contracts described here are based on the public wrapper, deterministic diagnostic helpers, helper tests, and native CSR sampler. The archived annotation draft supplies the broader motivation for direct annotation-dependent mixture priors.